Week 12 Milestone Report: Back End, Rules, and Security

Course: CS3320 Database Systems, Instructor Dr. Chibuzo Ukegbu
Student: Shritej Gunda (individual project)
Project: Factory Sherlock: Manufacturing Simulation, Quality Control, and Defect Investigation on MySQL
Date: [submission date]
Repository: https://github.com/shritej6/factory-sherlock
Commit or tag: [commit hash after push]

1. Completion status, incomplete items, and blockers

The Week 10 report listed five next steps. Their state on 2026-11-07, read from the integration branch `feature/completion-integration` at commit c978744:

Week 10 next step	Status	Evidence
1. Extend the API tests where the matrix marked scenarios as partly covered (503 body, cascade and SET NULL, quarantine rollback)	Done	Commit fba8bd8; backend/tests/api/test_error_contract.py, backend/tests/integration/test_foreign_keys.py, backend/tests/integration/test_transactions.py; docs/testing/test-matrix.md TM-01 to TM-19
2. Produce the data-flow diagram for the investigation workflow	Done, two stale sentences	docs/architecture/data-flow.md (five sequence diagrams; commit 2fddc42); see section 7, item 4
3. Demonstrate a database-backed workflow plus a controlled failure (an OPERATOR refused with 403)	Automated; live capture pending	backend/tests/api/test_investigations_api.py (`test_investigation_lifecycle_and_validation`: an operator opening an investigation receives 403; both investigation write routes require the Engineer set), frontend/e2e/roles.spec.ts (a viewer's direct POST receives 403 `AuthorizationError`)
4. Screenshots and videos; run the Docker path and the GitHub Actions workflows	Partly	Compose Smoke (2734f44) and End-to-End (3db7e11) workflows exist, five in total; the integration branch has not been pushed, so none has been observed since Week 10. Screenshots are still the nine from 2026-11-03 (docs/media/screenshots); no video exists (docs/media/videos/README.md)
5. Keep docs/ai_evidence current	Done	Commit f67ce19; docs/ai_evidence/ai-use-record.md rows 29 to 43; docs/ai_evidence/conversations/06-parallel-feature-integration.md

Also merged on 2026-11-07, beyond the Week 10 plan: list pagination (563c5e8), ORDER_OVERDUE alerts with a second migration (70b7589), the CSV inspection import (c7db63f), the login throttle (f3bf84c), bulk-insert simulation persistence (8e1dd9e), rule R7 (d1d9ead), the control chart query (abed3f8), status filters and order status counts (d22a1a7), and the review fixes described in sections 2 and 3.

Incomplete or in progress:

- The five workflows must be observed on the push of the integration branch; row TM-20 of the test matrix is still the reserved placeholder for the features added since Week 10.
- Screenshots do not show the import panel, the control chart, the overdue check, or the pagination controls; no walkthrough video has been recorded.
- The frontend does not recognise the 413 `payload_too_large` body, so an oversized upload shows the generic error text.
- [List any item that was planned for Week 12 and is not finished.]

Blockers: [describe any blocker, or write "none"].

2. Required technical artifacts and repository commit or tag

Artifact	Location
Running API	`make backend`, or `cd backend && uv run uvicorn app.main:app --reload --port 8000 --no-proxy-headers`; health at GET /api/v1/health; interactive docs at /api/docs outside production
API contract	docs/api/openapi.json (35 paths, 44 operations)
Data access	backend/app/core/database.py, backend/app/api/deps.py, backend/app/services/*.py, backend/app/repositories/*.py
Business rules	docs/requirements/business-rules.md (BR-01 to BR-116) with the enforcing module per rule
Validation and security	backend/app/schemas/*.py, backend/app/core/security.py, backend/app/core/throttle.py, backend/app/api/body_limit.py, backend/app/api/errors.py, SECURITY.md
Data-flow diagram	docs/architecture/data-flow.md; rendered diagrams in docs/media/diagrams/*.svg
Migrations	backend/migrations/versions (3a5e982c3679 tables, 383273c26ef7 alerts.production_order_id, 6c1f0d2a9b47 list ordering indexes); DDL snapshot database/schema/schema.sql
Tests	backend/tests (481 collected), frontend Vitest (118 tests in 25 files), frontend/e2e (9 tests in 5 spec files)
Repository commit or tag	[commit hash after push]

Running API. GET /api/v1/health (backend/app/api/routes/health.py) executes `SELECT 1` through the request session and answers `{"status":"ok","database":"ok"}`. Evidence to include in the PDF:

- [Terminal capture of `make backend` followed by `curl http://127.0.0.1:8000/api/v1/health`.]
- [Capture of POST /api/v1/auth/login as `engineer` and GET /api/v1/auth/me with the returned token.]
- [Capture of POST /api/v1/investigations for CNC-03 as `operator` returning 403 `AuthorizationError`, then as `engineer` returning 201 with its evidence cards.]

Data-access evidence. One engine per process and one session per request: `get_db_session` in backend/app/core/database.py yields the session, commits when the handler returns, rolls back when it raises, and closes in `finally`. The `DbSession` dependency in backend/app/api/deps.py uses `scope="function"` so a failed commit becomes an error response. No service commits: `grep -rn`

"\.commit()" backend/app finds only database.py and backend/app/jobs/check_overdue_orders.py, which owns its own transaction. Services call session.flush() to surface constraint failures inside the request transaction:

- backend/app/services/production_service.py create_batch flushes the order status change, the batch, the batch_materials rows, and one production_units row per unit together; _flush_or_conflict (backend/app/services/catalog_service.py) turns MySQL 1062 into 409 and, since 6635d8a, re-raises every other integrity error.
- backend/app/services/inspection_import.py _write_inspections runs chunked multi-row INSERTs inside a savepoint (session.begin_nested()), selects the new ids back by (unit_id, specification_id), and re-reads with with_for_update(read=True) to name rows another transaction recorded first.
- backend/app/services/alert_service.py reads overdue orders and their open alerts with SELECT ... FOR UPDATE; backend/app/repositories/query_runner.py runs the 14 SQL files under database/queries through SQLAlchemy text() with named binds; backend/tests/integration/test_transactions.py fails the last bulk insert of a simulation run and asserts that nothing remains.

Business rules. Every rule in docs/requirements/business-rules.md names its enforcing layer (MySQL constraint, domain state machine, service check, schema validation, or domain function). By group:

Range	Area	Enforcing modules
BR-01 to BR-06	Identity and access	UNIQUE on users; backend/app/services/auth_service.py; backend/app/api/deps.py (require_roles; get_current_user reloads the row)
BR-10 to BR-17	Reference data	backend/app/schemas/catalog.py (CODE_PATTERN), UNIQUE and CHECK constraints, ENUM columns, catalog_service
BR-20 to BR-24	Machines	backend/app/domain/state_machines.py (MACHINE_TRANSITIONS), catalog_service, backend/app/simulation/machine_state.py
BR-30 to BR-40	Production orders	ORDER_TRANSITIONS, production_service, CHECK ck_order_completed_within_planned
BR-50 to BR-58	Batches and units	production_service, BATCH_TRANSITIONS, UNIQUE batch_number and serial_number, ON DELETE RESTRICT on the three child tables
BR-60 to BR-69	Quality	backend/app/domain/quality_rules.py (evaluate_measurement, classify_defect, defect_rate), backend/app/services/quality_service.py, UNIQUE uq_inspection_unit_spec
BR-70 to BR-78	Investigations	backend/app/services/investigation_service.py, INVESTIGATION_TRANSITIONS, backend/app/investigation/rules.py (RuleThresholds)
BR-80 to BR-85	Traceability and quarantine	backend/app/services/traceability_service.py; QUARANTINED is terminal and no release endpoint exists
BR-90 to BR-95	Simulation	backend/app/schemas/simulation.py, backend/app/simulation/config.py and engine.py, backend/app/services/simulation_service.py
BR-100 to BR-104	Overdue order alerts	backend/app/services/alert_service.py (OPEN_ORDER_STATES, days_late, overdue_severity, CRITICAL_AFTER_DAYS), backend/app/api/routes/analytics.py (no request body), fk_alerts_production_order ON DELETE SET NULL

Range	Area	Enforcing modules
BR-110 to BR-116	Inspection import	backend/app/services/inspection_csv.py (header, field parsers, in-file duplicates), backend/app/services/inspection_import.py (reference resolution, all-or-nothing write), `uq_inspection_unit_spec` as the final guard

Validation and security. SECURITY.md lists every control with its file; the ones that changed this milestone:

- Every request schema in backend/app/schemas sets `extra="forbid"`, so unknown fields return 422 with per-field messages. Paging accepts `limit` 1 to 500 and `offset` 0 to 1,000,000.
- Roles: the `Operator` and `Engineer` dependencies in deps.py; ADMIN is in both sets; reads need any active user. The role in the token is informational, the reloaded user row decides (`test_the_database_role_decides_not_the_token`).
- Passwords are hashed with bcrypt directly (backend/app/core/security.py). Tokens are HS256 JWTs carrying `sub`, `role`, `iat`, and `exp`, lifetime `JWT_ACCESS_TOKEN_MINUTES` (default 120); `JWT_SECRET_KEY` must be at least 32 characters and is declared `repr=False`.
- Login throttle (backend/app/core/throttle.py, redesigned in 18a5903). Before the password is verified only the account bucket is checked: the user id when the username matches a row, otherwise the username normalised with NFKD, combining marks removed, and casefolded, so `admin`, `ADMIN`, and `ädmin` share a bucket. `check` reserves a pending slot under the lock that `record_failure` or `record_success` converts or releases, so concurrent requests cannot exceed `LOGIN_MAX_FAILURES` (default 5) inside `LOGIN_WINDOW_SECONDS` (default 900). The client address is counted only after a failed verification and turns that failure from 401 into 429. Guarantees, each pinned by a test: a correct password is never rejected because of the address; five failures as `engineer` do not lock `operator` or `viewer` out from the same address; 20 threads on a barrier admit exactly 5; 429 is returned whether or not the username exists; success clears the account counter but not the address counter. Every self-started uvicorn passes `--no-proxy-headers` so the address is the TCP peer.
- Request body caps (backend/app/api/body_limit.py, pure ASGI, innermost middleware): 1 MiB + 64 KiB for POST /api/v1/inspections/import, 64 KiB elsewhere. A declared `Content-Length` over the cap is answered 413 before authentication; a chunked body is counted as it streams and cut off once over. The body has the standard shape with `error: "payload_too_large"` and `details.limit_bytes`.
- Error contract (backend/app/api/errors.py). Every error body is `{error, message, details, request_id}` and the same id is in the `X-Request-ID` header (backend/app/api/request_id.py). Pure `classify_*` functions hold the MySQL errno mapping: 1062 to 409; 1452 to 422 "references a row that does not exist"; 3819 to 422 with `details.constraint` parsed from the driver message; 1406 to 422 "too long for its column"; 1205 and 1213 to 409 "retry"; 2002, 2003, 2006, 2013 and pool timeouts to 503 "not reachable"; other `OperationalError` to 503 "unavailable"; everything else to 500 without SQL text or driver messages. JSON logs (backend/app/core/logging.py) keep an allowlist of fields and never carry usernames, passwords, or request bodies.

Data-flow diagram. docs/architecture/data-flow.md draws the five write paths as Mermaid sequence diagrams with participants named after modules in backend/app; the request pipeline is in docs/architecture/component-design.md under "Request flow". Eight rendered diagrams are in

docs/media/diagrams (system-architecture.svg, backend-components.svg, investigation-workflow.svg, er-diagram.svg, and four more).

Tests. `uv run pytest --collect-only -q` in backend/ at c978744 on 2026-11-07 collected 481 tests: 254 unit, 174 API, 53 integration. `pnpm -s test` in frontend/ reported 25 files and 118 tests passed; frontend/e2e holds 5 Playwright spec files with 9 tests. Files that prove the rules in this report, with collected cases:

Rule or control	Test files
Login throttle	backend/tests/unit/test_login_throttle.py (21, pinned clock), backend/tests/api/test_login_throttle_api.py (10)
Error contract and errno mapping	backend/tests/unit/test_error_mapping.py (19), backend/tests/api/test_error_contract.py (10), backend/tests/api/test_auth_and_errors.py (30)
Body caps	backend/tests/unit/test_body_limit.py (5), backend/tests/api/test_body_limit_api.py (4)
BR-110 to BR-116 import	backend/tests/unit/test_inspection_csv.py (35), backend/tests/api/test_inspection_import.py (20, including the race and the sample file)
BR-100 to BR-104 overdue alerts	backend/tests/unit/test_alert_rules.py (11), backend/tests/api/test_alert_checks_api.py (8), backend/tests/integration/test_alert_order_link.py (6, two-session concurrency, SET NULL, job exit codes)
State machines, quality rules, schemas	backend/tests/unit/test_state_machines.py (79), backend/tests/unit/test_quality_rules.py (15), backend/tests/unit/test_schemas.py (24)
Transactions and foreign keys	backend/tests/integration/test_transactions.py (25), backend/tests/integration/test_foreign_keys.py (14), backend/tests/integration/test_list_ordering_indexes.py (2)
Pagination and filters	backend/tests/api/test_pagination_api.py (12), backend/tests/api/test_status_filters_api.py (18)

3. Decisions, alternatives, assumptions, constraints, and rationale

Decision	Alternative considered	Rationale
Throttle: account bucket before verification, address only after a failure	Check the client address first, as the first version did	Behind the Vite proxy every browser arrives from 127.0.0.1, so an address-first check let five wrong passwords from one person lock every demo account, and uvicorn's default trust of `X-Forwarded-For` from loopback let a client pick its own address. The account bucket is the lockout that matters; the address counter still slows a run through many usernames without rejecting a correct password.
Remove `as_of` from the HTTP overdue check; keep `--as-of` on the job only	Keep the optional body and restrict it to ADMIN	Any Operator could post a future date to raise alerts for orders that are not late, or a past date to back-date the feed. The server clock is the only honest check date for a web client; replaying a past check is a command line task with its own log line.

Decision	Alternative considered	Rationale
Bulk import path: validate every row in memory, then chunked multi-row INSERTs in a savepoint	Call `quality_service.record_inspection` once per row	Per-row recording cost 10507 statements and 1.47 s for the 5000-row maximum on the benchmark database; the bulk path takes 15 statements and 0.11 s with the same 201 or 422 all-or-nothing contract (docs/quality/inspection-import.md). Both paths share `evaluate_measurement`, so pass or fail cannot drift between manual and imported entries.
Pure ASGI body cap middleware with per-route limits and the standard JSON body	Starlette's built-in `RequestBodyLimitMiddleware` (`app.max_body_size`)	The built-in limiter applies one cap to every route and answers plain text; the import needs 1 MiB plus multipart framing while a JSON route needs 64 KiB, and every error body in this API carries `request_id`.
`GET /production-orders/status-counts` with one GROUP BY, plus repeatable `status` filters applied in SQL	Client fan-out: one list request per status, and filtering 500-row pages in the browser	The Overview made six single-row requests for one widget and loaded every investigation with its evidence (about 9 MB) to count open cases; counts were wrong once a table passed 500 rows. One GROUP BY returns every status, zero included, and `total` reflects the filter.
`SELECT ... FOR UPDATE` on both reads of the overdue check	A functional unique index on `alerts` for open ORDER_OVERDUE rows	MySQL has no partial unique index; a functional index on a CASE expression was probed on MySQL 8.4 and works, but it is a schema change for a later migration. The row locks already serialise overlapping checks; the second locking read is needed because under REPEATABLE READ a plain SELECT would use the snapshot opened before the lock wait.

Assumptions and constraints: a single API process (the throttle store is in-process memory and documented as such); MySQL 8.4 with its default `sql_mode` including `ONLY_FULL_GROUP_BY`; no HTTPS termination in development; demo accounts share `SEED_DEMO_PASSWORD`. Known limitations are listed in SECURITY.md.

4. AI help: what, why, accepted, revised, rejected, and verification

Tool: Claude Code (Anthropic), used for implementation, review passes, and documentation drafts during this milestone. Every change ran through the automated gates before commit: ruff check and format, mypy, pytest, eslint, tsc, vitest, Playwright, and `make check-docs`, and the student read each diff before committing. The record is in docs/ai_evidence: ai-use-record.md (rows 29 to 43 cover this milestone) and conversations/06-parallel-feature-integration.md.

[State in your own words which suggestions from this milestone you accepted, which you revised, and which you rejected, with the verification you ran for each. Candidates from the record: the address-first throttle (revised after review, row 36 and commit 18a5903), the per-row import write path (revised to the bulk path, commit 1adad00), the `as_of` body on the overdue endpoint (rejected, commit ef47abe), and the downgrade of migration 6c1f0d2a9b47.]

5. Lessons learned at this milestone

- A control that is correct in isolation can fail in its deployment context. The first throttle counted the address before verification; review found that behind the Vite proxy every browser shares 127.0.0.1, so one user's failures locked every account (18a5903). The fix was a design change, not a parameter.
- A list contract change reaches every client at once. The pagination envelope (563c5e8) broke eight API tests on branches written in parallel against bare arrays, plus the sample-file import test and the end-to-end helper (7bca7ac, a95df14).
- MySQL couples indexes to foreign keys. Creating `ix_inspections_spec_time` made MySQL drop the implicit `specification_id` index, and the downgrade fails with error 1553 unless that index is recreated before the composite one is dropped (6c1f0d2a9b47). Migrations must be run up, down, and up again on the real server.
- REPEATABLE READ hides concurrent commits from a plain SELECT. The import race check and the overdue check both needed locking reads.
- Measure before optimising. The import benchmark (scripts/development/benchmark_inspection_import.py) showed the cost was round trips, not parsing: 10507 statements to 15 for the same file.
- [Add one lesson in your own words.]

6. Support or skill development needed for the next stage

- Observing the Compose Smoke and End-to-End workflows requires a push of the integration branch; the first run ids will be recorded in docs/testing/test-matrix.md.
- [State any instructor support or skill development you need for Week 13, or write "None at this time".]

7. Next-step plan (Week 13)

1. Push `feature/completion-integration`, observe the five workflows (Backend CI, Frontend CI, Docs Lint, Compose Smoke, End-to-End), record run ids in docs/testing/test-matrix.md, and replace the reserved TM-20 row with one row per feature added since Week 10.
2. Capture screenshots of the import panel with row errors, the control chart, the overdue check, the login 429 message, and the paginated tables; record the complete system walkthrough video from docs/media/media-plan.md and add its row to docs/media/videos/README.md.
3. Teach the frontend `describeError` the 413 `payload_too_large` body, with a Vitest case.
4. Correct the stale statements found while writing this report: docs/architecture/data-flow.md (import section still says `record_inspection` per row; overdue section still shows the `as_of` body), docs/requirements/non-functional-requirements.md NFR-1.11 (pre-redesign throttle), docs/testing/testing-strategy.md (says no end-to-end test exists), frontend/e2e/investigation.spec.ts `RULE_CODES` (R1 to R6; R7 exists).
5. Decide on the functional unique index for open ORDER_OVERDUE alerts (BR-103) and, if adopted, add a fourth reversible migration with a round-trip test.
6. Prepare the Week 13 integration evidence: docs/api/openapi.json regenerated by `make export-openapi`, the Playwright HTML report, and the interface walkthrough against the demo scenario.

8. Personal reflection (Shritej Gunda)

[Write 150 to 250 words in your own voice. Replace each prompt.]

[Which business rule or security control did you find hardest to get right, and how did you confirm it behaves as specified (which test, which capture)?]

[Explain the transaction boundary of one request (where the session opens, flushes, commits, rolls back) in your own words as you would in the defense.]

[Describe one AI suggestion from this milestone that you revised or rejected, how you found the problem, and what you learned about MySQL or about web security from it.]

[What do you need to practice before the Week 13 front end and integration defense?]

9. Personal-work summary

[List the artifacts you personally produced or verified for this milestone, one line each, with the file path or screen name.]